

YAPAY ZEKA DİL AJANLARI YARIŞMASI

Katılım Bankacılığı Finansal Metin Madenciliği

PROJE DOKÜMANTASYONU

Harput Finans

Yapay Zekâ Destekli Katılım Finans Karar Destek ve Bilgi Çıkarım Platformu

Takım: HarputAI

TAKIM ÜYELERİ

Yusuf Şimşek • Ali Eren Kaya • Yasin Muaz Yüksel • Tayyip Şimşek

Ağustos 2026

Yönetici Özeti

Katılım bankalarının (kâr payı esasına göre çalışan bankaların) sunduğu finansman ürünleri ve kampanyalar, her bankanın kendi internet sitesinde farklı biçimde anlatılıyor. Bu yüzden bir kullanıcının 10 bankayı karşılaştırıp kendisine en uygun seçeneği bulması oldukça zor ve zaman alıcı bir iş.

Bu sorunu çözmek için HarputAI takımı olarak **Harput Finans** adını verdiğimiz bir platform geliştirdik.

Harput Finans yalnızca bir sohbet botu değildir; birbirinden farklı görevleri yerine getiren birçok bileşenden oluşan bir sistemdir. Özetle şunları yapar:

- Bankaların sitelerindeki serbest, düzensiz yazılmış metinleri yapay zeka yardımıyla düzenli, karşılaştırılabilir bir veri tablosuna dönüştürür.
- Kullanıcı doğal dilde bir soru sorduğunda (“Kuveyt Türk ile Ziraat'i karşılaştır” gibi), bu düzenli veriden doğru cevabı bulup kaynağıyla birlikte sunar.
- Kullanıcının ihtiyacına göre (tutar, vade, amaç) uygun finansman seçeneklerini önerir.
- Veri tabanındaki her ürün veya kampanya kaydından, tek tuşla doğrudan yapay zeka asistanına soru sorulmasını sağlar.

En önemli tasarım kararımız şudur: **yapay zeka modeli hiçbir zaman sayı uydurmuyor veya tahmin yapmıyor**. Sayısal karşılaştırmaları her zaman sabit, güvenilir bir program yapıyor; yapay zeka yalnızca bu sonucu anlaşılır bir dille anlatıyor.

Bu doküman, sistemin en baştan bugünkü hâline kadar geçirdiği teknik süreci; mimarisini, veri setini, karşılaşılan gerçek problemleri ve çözümlerini, kurulumunu ve örnek çıktılarını adım adım açıklamaktadır. Her bölümün başında, teknik bilgisi olmayan bir okuyucunun da anlayabileceği kısa bir “Kısaca” özeti bulunmaktadır. Teknik terimler için bir sonraki sayfadaki Terimler Sözlüğü kullanılabilir.

Terimler Sözlüğü

Bu bölümde, dokümanın ilerleyen kısımlarında geçen teknik terimlerin sade Türkçe açıklamaları yer almaktadır. Herhangi bir terimle karşılaştığınızda bu sayfaya geri dönebilirsiniz.

Terim	Açıklama
Yapay Zeka (AI)	Bilgisayarların, insan zekâsı gerektiren görevleri (anlama, öğrenme, karar verme) yerine getirebilmesini sağlayan teknoloji alanı.
Büyük Dil Modeli (LLM)	İnsan dilini anlayıp metin ve kod üretebilen, çok büyük miktarda veriyle eğitilmiş bir yapay zeka modeli türü.
NLP (Doğal Dil İşleme)	Bilgisayarların insan dilini anlaması, yorumlaması ve üretmesiyle ilgilenen yapay zeka alt alanı.
Program Sentezi	Bir yapay zeka modelinin, bir görevi çözecek bilgisayar kodunu kendisinin yazması.
Regex (Düzenli İfade)	Bir metindeki belirli kalıpları aramak için kullanılan sabit bir arama kuralı.
JSON	Bilgilerin ad-değer çiftleri hâlinde, düzenli ve bilgisayarın kolayca okuyabileceği şekilde saklandığı bir veri formatı.
Intent (Niyet)	Kullanıcının sorusunun arkasındaki asıl amaç veya istek türü (örn. bilgi almak, karşılaştırmak, öneri istemek).
Intent Detection (Niyet Tespiti)	Kullanıcının sorduğu sorunun niyetini otomatik olarak anlayıp sınıflandırma adımı.
Ajan / Tool (Araç)	Belirli bir görevi her zaman aynı mantıkla, güvenilir biçimde yerine getiren yazılım bileşeni.
Resolver (Çözümleyici)	Kullanıcının doğal dil ifadesini, veri tabanındaki gerçek bir kayıtlarla (örn. belirli bir banka veya kampanya) eşleştiren bileşen.
Deterministik	Aynı girdi için her zaman aynı çıktıyı üreten, rastgele davranmayan (yapay zekanın aksine öngörülebilir) sistem veya kod.
AST (Abstract Syntax Tree)	Bir bilgisayar kodunun, çalıştırılmadan önce yapısal olarak incelenebildiği temsili; kodun güvenli olup olmadığını denetlemek için kullanılır.
Subprocess (Alt Süreç)	Bir programın, ana sistemden ayrı, izole ve korumalı bir ortamda çalıştırılan kopyası.
Context Window (Bağlam Penceresi)	Bir yapay zeka modelinin tek seferde okuyup işleyebildiği azami metin uzunluğu.
Token	Metnin model tarafından işlenirken bölündüğü küçük parçalar; modelin çıktı uzunluğu genellikle token sayısı ile ölçülür.
Temperature	Modelin çıktı üretirken ne kadar değişken/rastgele davranacağını belirleyen ayar; düşük değer daha tutarlı ve öngörülebilir sonuç anlamına gelir.
Truncation (Kesilme)	Modelin cevabının, izin verilen uzunluk sınırı nedeniyle yarıda kesilmesi.
Grounding (Kaynağa Dayalı Üretim)	Modelin verdiği cevabı, kendi hayal gücüyle değil, önceden doğrulanmış gerçek verilerle sınırlı tutması.

Grounding Compaction (Bağlam Sıkıştırma)	Çok büyük miktardaki veriyi, modelin verimli işleyebileceği kısa ve öz bir özete dönüştürme işlemi.
Precision / Recall / F1	Bir modelin ne kadar doğru çalıştığını ölçmek için kullanılan istatistiksel başarı ölçütleri.
Latency (Yanıt Süresi)	Bir sorunun sisteme gönderilmesinden, cevabın kullanıcıya ulaşmasına kadar geçen süre.
Canonicalization (Standartlaştırma)	Farklı şekillerde ifade edilen aynı bilginin, sistem içinde tek ve tutarlı bir biçime dönüştürülmesi.
Session (Oturum)	Bir kullanıcının sistemle olan tek bir kullanım sürecini temsil eden, diğer kullanıcılardan izole edilmiş veri alanı.
RLock (Yeniden Girişli Kilit)	Aynı anda birden fazla işlemin aynı veriye erişmesini güvenli şekilde kontrol eden bir programlama mekanizması.
GPU / VRAM	Yapay zeka modellerinin çalıştırıldığı özel işlemci (GPU) ve bu işlemcinin bellek kapasitesi (VRAM).
LangGraph	Yapay zeka ajanının hangi adımdan hangi adıma geçeceğini (örn. hangi aracın çağrılacağını) kontrollü biçimde yöneten bir orkestrasyon çatısı.
Quick Tunnel	Yerel bir bilgisayarda çalışan uygulamayı geçici bir internet adresi üzerinden dışarıya açan araç (örn. Cloudflare Quick Tunnel).
Ground Truth (Referans Değer)	Bir modelin çıktısının doğruluğunu ölçmek için kullanılan, önceden bilinen/doğrulanmış “doğru cevap” kümesi.
Baseline (Kıyas Noktası)	Yeni bir yöntemin performansını karşılaştırmak için kullanılan, daha basit veya alternatif bir referans yöntem.
Frozen Test Seti (Dondurulmuş Test Seti)	İçeriği bir kez belirlenip değiştirilmeyen, aynı haliyle tekrar tekrar kullanılan test soruları/vakaları kümesi; farklı ölçümlerin adil karşılaştırılmasını sağlar.
Regresyon Testi	Bir hata düzeltildikten sonra, sistemin hem o hatayı çözdiğünü hem de başka bir şeyi bozmadığını doğrulamak için aynı test setinin yeniden çalıştırılması.
Jaccard Benzerliği	İki sonuç kümesinin ne kadar örtüştüğünü ölçen bir istatistik; burada aynı metnin farklı çalıştırmalarda ne kadar tutarlı sonuç ürettiğini ölçmek için kullanılmıştır.

İçindekiler

Yönetici Özeti.....	2
Terimler Sözlüğü.....	3
İçindekiler	5
1. Sistem Mimarisinin Genel Açıklaması ve Veri Akışının Özeti	6
2. Kullanılan NLP Yaklaşımının Açıklaması	8
3. Kullanılan Veri Seti ve Açıklaması	9
4. Veri Ön İşleme Adımları	10
5. Model veya Kural Yapısının Açıklaması	11
6. Farklı Bankalara Ait Benzer Ürünlerin Nasıl Karşılaştırıldığının Açıklaması	13
7. Projenin Çalıştırılması için Adım Adım Talimatlar	15
8. Karşılaşılan Problemler ve Çözüm Yaklaşımları.....	17
9. Model Çıktılarının Örnekleri	19
10. Modelin Performansını Değerlendirmek için Kullanılan Yöntemler (Varsa)	21

1. Sistem Mimarisinin Genel Açıklaması ve Veri Akışının Özeti

Kısaca: Harput Finans; bankaların dağınık verilerini toplayıp düzenleyen ve bunu bir yapay zeka asistanı aracılığıyla kullanıcıya sunan, birden fazla bileşenden oluşan uçtan uca bir sistemdir. Yapay zeka hiçbir zaman tek başına karar vermez; her zaman doğrulanabilir, sabit bir veri katmanına dayanır.

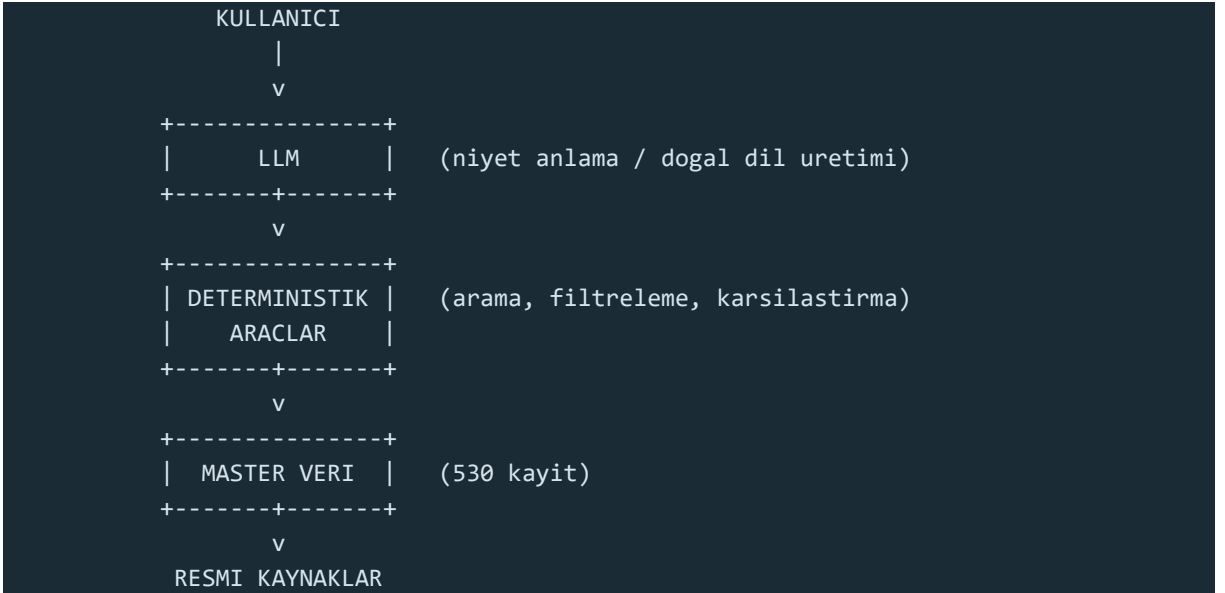
Harput Finans, Türkiye'de faaliyet gösteren katılım bankalarının bireysel finansman ürünleri ve kampanyalarına ilişkin bilgileri tek bir platform altında birleştiren, bu verileri yapılandıran, karşılaştıran ve doğal dil üzerinden erişilebilir hâle getiren yapay zekâ destekli bir finansal karar destek sistemidir.

Sistem yalnızca bir sohbet botu değildir; aşağıdaki görevleri yerine getiren birbirinden farklı bileşenlerden oluşur:

- Katılım bankalarından veri toplama
- Verilerin normalize edilmesi
- Finansman ve kampanya kayıtlarının ortak şemada saklanması
- Serbest finansal metinlerden yapılandırılmış bilgi çıkarımı
- Finansman ürünlerini ve kampanyaları sorgulama
- Bankalar arası kriter bazlı karşılaştırma
- Kullanıcının ihtiyaçlarına göre finansman önerisi
- Doğal dil sorgularını uygun araçlara yönlendiren yapay zekâ ajanı
- Kaynak gösterimi ve banka bazlı veri gezgini
- Web tabanlı kullanıcı arayüzü, REST API ve yerel LLM altyapısı

Temel Mimari Prensibi

Projenin temel yaklaşımı, büyük dil modelinin (LLM) tek başına bütün kararları vermesi yerine LLM + deterministik araçlar + doğrulanabilir veri + kaynak gösterimi mimarisini kullanmaktır.



Uçtan Uca Veri Akışı

1. Veri Toplama: Bankaların kamuya açık resmî web sayfalarından finansman ürünü ve kampanya sayfaları, her banka için ayrı geliştirilen scraper'larla taranır.

2. Ham Metin Elde Etme: Listeleme ve detay sayfalarından ham metin çıkarılır; kaynak URL her kayıtla birlikte saklanır.
3. Dinamik Bilgi Çıkarımı: Ham metin extraction modeline (qwen3-coder-next) gönderilir; model metne özel bir Python ayrıştırma kodu üretir ve 9 alanlık yapılandırılmış JSON elde edilir.
4. Şemaya Yerleştirme: Çıkarılan alanlar, 18 alanlık ortak master finans şemasına yerleştirilir.
5. Depolama: Yapılandırılmış kayıtlar merkezi bir veri deposunda tutulur (530 kayıt: 116 finansman ürünü + 414 kampanya, 10 banka).
6. Kullanıcı Sorgusu: Kullanıcı doğal dilde bir soru sorduğunda, Intent Detection adımı sorunun niyetini sınıflandırır.
7. Araç Çağrısı: Sınıflandırılan niyete göre doğru deterministik araç (finansman ya da kampanya aracı) çağrılır.
8. Kaynaklı Yanıt: Aracın döndürdüğü deterministik sonuç, agent modeli tarafından doğal dile çevrilir ve kaynak bağlantılarıyla birlikte kullanıcıya sunulur.

REST API Uç Noktaları

Uç Nokta	Görevi
GET /	Servisin ana sayfası / temel bilgi.
POST /extract	Serbest finansal metinlerden yapılandırılmış alan çıkarımı gerçekleştirir.
POST /chat	Agent ile deterministik finansman/kampanya araçlarını çalıştırır.
POST /chat/ui	Web arayüzü için güvenli şekilde zenginleştirilmiş (rozet, kaynak vb.) yanıt döndürür.
POST /chat/reset	Oturuma ait konuşma durumunu (state) temizler.
GET /health	Servisin çalışma durumunu kontrol etmek için kullanılır.

Çalışma Ortamı Bileşenleri

Bileşen	Açıklama
API Runtime	FastAPI / Uvicorn
Agent Modeli	qwen3.5:35b-a3b-q8_0
Extractor Modeli	qwen3-coder-next
Model Çalışma Ortamı	Ollama · Yerel çıkarım (Local Inference)
Ajan Mimarisi	LangGraph + Deterministik Yönlendirme (Routing)
Donanım	Geliştirme/demo ortamında yüksek VRAM'e sahip NVIDIA GPU kullanılmıştır.

2. Kullanılan NLP Yaklaşımının Açıklaması

Kısaca: Sistemde tek bir yapay zeka yöntemi kullanılmıyor; bilgiyi metinden çıkarmak için “kod yazan” bir model, kullanıcının ne istediğini anlamak için niyet sınıflandırma, cevabı ifade etmek için ise kaynağa dayalı metin üretimi bir arada kullanılıyor.

a) Program Sentezi Tabanlı Bilgi Çıkarımı

Klasik bilgi çıkarımı yaklaşımlarının (sabit regex kuralları veya doğrudan alan tahmini yapan modeller) aksine, sistemimiz metni doğrudan yapılandırmak yerine, o metne özgü bir ayrıştırma programı (Python/regex kodu) üretir.

Bu yaklaşımın gerekliliği somut örneklerle ortaya çıkmıştır: “%80” ifadesinin finansman oranı mı, kâr payı mı, yoksa bir indirim oranı mı olduğu; “24 ay” ifadesinin vade mi, taksit sayısı mı, yoksa kampanya süresi mi olduğu, yalnızca regex ile güvenilir biçimde ayırt edilememiştir.

Bunun üzerine extraction işlemi, “model bilgiyi tahmin etsin” yerine “model bilgiyi çıkaracak kodu yazsın” şeklinde yeniden tasarlanmıştır. Bu, klasik NLP taksonomisinde program sentezi (program synthesis) destekli bilgi çıkarımına karşılık gelir.

b) Niyet Sınıflandırma / Yönlendirme (Intent Classification & Routing)

Kullanıcının doğal dildeki sorusu, önceden tanımlanmış niyet kategorilerinden birine sınıflandırılır:

Intent	Anlamı
PRODUCT_INFO	Belirli bir finansman ürünü hakkında bilgi isteği
CAMPAIGN_INFO	Belirli bir kampanya hakkında bilgi isteği
PRODUCT_COMPARE	Bankalar arası finansman ürünü karşılaştırma isteği
CAMPAIGN_COMPARE	Bankalar arası kampanya karşılaştırma isteği
PERSONAL_RECOMMENDATION	Kullanıcının ihtiyacına göre öneri isteği
FOLLOW_UP	Önceki soruya bağlı, bağlamı koruyan takip sorusu
GENERAL_CHAT	Genel sohbet / kapsam dışı sorular

Bu sınıflandırma, hangi aracın (tool) çağrılacağını belirleyen bir yönlendirme mekanizması olarak çalışır. Önemli bir tasarım detayı: LLM tarafından üretilen intent sonucu doğrudan kabul edilmez; sisteme kabul edilmeden önce bir dizi “contract guard” (sözleşme denetimi) kontrolünden geçirilir. Böylece modelin kendi çıktısı bile kör güvenle kullanılmaz.

c) Kaynağa Dayalı Doğal Dil Üretimi (Grounded Natural Language Generation)

Araçlardan dönen deterministik, yapılandırılmış sonuç, agent modeli tarafından akıcı bir doğal dil yanıtına dönüştürülür. Buradaki ilke açıktır: LLM bilgiyi bulmaz, yalnızca bulunan bilgiyi anlatır. Bu, serbest üretim değil, kaynağa sabitlenmiş (grounded) bir üretimdir.

Özet

Sistem; kural tabanlı yöntemin (regex) yetersiz kaldığı noktada üretici bir modelle program sentezine geçmiş, ardından bir sınıflandırma/yönlendirme katmanı ile deterministik araçları tetikleyen, son adımda ise üretimi kaynağa sabitleyen hibrit bir mimari kullanmaktadır.

3. Kullanılan Veri Seti ve Açıklaması

Kısaca: Sistemin kullandığı bilgi tabanı, 10 gerçek katılım bankasının herkese açık resmî internet sitelerinden toplanmış 530 gerçek kayıttan oluşmaktadır.

Veri seti, 10 katılım bankasının kamuya açık resmî web sitelerinden toplanan gerçek finansman ürünü ve kampanya kayıtlarından oluşmaktadır. Toplam 530 kayıt bulunmaktadır: 116 finansman ürünü ve 414 kampanya kaydı.

Banka	Finansman	Kampanya	Toplam
Ziraat Katılım	20	87	107
Kuveyt Türk	30	73	103
Türkiye Emlak Katılım	12	63	75
Albaraka Türk	17	46	63
TOM Katılım	3	50	53
Dünya Katılım	6	43	49
Vakıf Katılım	8	26	34
Türkiye Finans	16	15	31
Hayat Finans	3	11	14
Adil Katılım	1	0	1
TOPLAM	116	414	530

Temel Ortak Veri Şeması (18 Alan, 4 Kategori)

Kategori	Alanlar
Kimlik	banka, kayıt_turu, urun_adi, urun_kategorisi
Finans	kar_payi_orani, finansman_orani, finansman_tutari, vade, taksit_sayisi, masraf_bilgisi
Kampanya	kampanya_turu, kampanya_avantajı, kampanya_suresi, hedef_kitle
Kaynak	para_birimi, kosullar, kaynak_url, ham_metin

Bu 18 alan, tüm banka kayıtlarında ortak kullanılan temel şemayı oluşturmaktadır. Sistem içerisinde karşılaştırma, kampanya sınıflandırma ve tarih analizi amacıyla kategori, fayda türü, tarih durumu ve karşılaştırma metrikleri gibi ek türetilmiş alanlar da oluşturulmaktadır.

Sistemin temel prensiplerinden biri şudur: **kaynak veride bulunmayan bilgi tahmin edilmez**. Bu nedenle bilinmeyen alanlar kullanıcı arayüzünde “Belirtilmemiş” olarak gösterilmektedir. Her kayıta kaynak URL saklandığından, veri setinin izlenebilirliği ve doğrulanabilirliği sağlanmaktadır.

4. Veri Ön İşleme Adımları

Kısaca: Bankaların sitesindeki ham, düzensiz metinler; her banka için özel geliştirilen bir toplama akışıyla adım adım işlenerek düzenli, karşılaştırılabilir bir veri tablosuna dönüştürülüyor.

Banka Bazlı Web Scraping Süreci

Bankaların web sitelerinin HTML yapıları birbirinden tamamen farklı olduğu için tek bir genel scraper kullanmak yerine banka bazlı scraper mimarisi tercih edilmiştir. Genel veri toplama süreci şu şekildedir:

```
Banka ana sayfası
  v
Finansman / Kampanya liste sayfası
  v
Urun bağlantılarının bulunması
  v
Her ürünün detay sayfasının açılması
  v
Başlık + kategori + açıklama + kaynak URL
  v
Ham kayıt
  v
Normalize kayıt
  v
Master veri kümesi
```

Bu süreçte yalnızca ürün adı değil; detay sayfasının URL'si, kategori, açıklamalar, kampanya avantajları, koşullar ve ham metin de korunmuştur. Bunun önemli bir sebebi, sonraki aşamalarda veri çıkarımı yapılırken yalnızca normalize edilmiş alanlara değil, gerektiğinde orijinal ham içeriğe de erişebilmektir.

Veri Birleştirme ve Master Veri Kümesi

Her bankadan gelen kayıtların yapısı farklı olduğundan verilerin doğrudan tek bir veri kümesinde kullanılması mümkün değildir. Bu nedenle bütün kayıtlar ortak, 18 alanlı bir master şemaya dönüştürülmüştür (bkz. Bölüm 3). Bu şema hem finansman hem kampanya kayıtlarının aynı sistem içinde tutulabilmesini sağlar; örneğin bir finansman ürünüde kampanya alanları boş kalabilirken, bir kampanya kaydında finansman oranı bulunmayabilir.

Veri Kalite Kontrolleri

Scraping işlemi tamamlandıktan sonra kayıtlar doğrudan kullanılmamış; aşağıdaki kontrollerden geçirilmiştir:

- Duplicate (tekrarlayan) URL kontrolü
- Duplicate kayıt kontrolü
- Şema doğrulaması
- Zorunlu alan kontrolü
- Banka adlarının normalize edilmesi
- Kayıt türünün kontrol edilmesi
- Kaynak URL kontrolü
- Ham metnin korunması

Bu sayede farklı bankalardan gelen kayıtlar ortak bir yapıda birleştirilerek tek bir master veri kümesi oluşturulmuştur.

5. Model veya Kural Yapısının Açıklaması

Kısaca: Biri metinden bilgi çıkarmak, diğeri kullanıcıyla konuşup doğru cevabı bulmak için olmak üzere iki farklı yapay zeka modeli kullanılıyor; finansal hesaplamaları ise yapay zekaya değil, sabit ve güvenilir bir programa yaptırıyoruz. Ayrıca, aynı anda iki büyük modeli bellekte tutmamak için modeller ihtiyaca göre bellekten yüklenip çıkarılıyor.

a) Extraction Modeli — qwen3-coder-next

- Görevi: Serbest finansal metinden, metne özel bir Python/regex ayrıştırma kodu üretmek.

Ayar	Değer
Context Window	32.768 token
Temperature	0.05 (düşük tutarlılık/yüksek öngörülebilirlik için)
Top-p	0.95
Max Output	8.192 token
Thinking	false — model ara “düşünme” adımı göstermeden doğrudan sonuç üretir

- Güvenlik: Üretilen kod, çalıştırılmadan önce AST (Abstract Syntax Tree) tabanlı statik analizden geçirilir. Tehlikeli import'lar, sistem komutları, dosya sistemi üzerinde istenmeyen işlemler, process çalıştırma ve izin verilmeyen fonksiyonlar tespit edilip reddedilir.
- İzole Çalıştırma: Kabul edilen kod, ayrı bir subprocess içinde, timeout (süre sınırı) ile çalıştırılır; extraction programı beklenmeyen şekilde çalışsa bile ana uygulama etkilenmez.
- Çıktı Sözleşmesi: Model her zaman 9 alanlık sabit bir yapı döndürür (kâr payı oranı, finansman oranı, finansman tutarı, vade, taksit sayısı, masraf bilgisi, hedef kitle, para birimi, koşullar). Public /extract API'si yalnızca bu 9 alanı döndürür; modelin ürettiği Python kodu, güvenlik denetim sonucu ve diğer iç (debug) bilgiler kullanıcıya gösterilmez.

b) Agent (Ajan) Modeli — qwen3.5:35b-a3b-q8_0

Ollama üzerinde yerel olarak çalıştırılan bu model, kullanıcının doğal dildeki sorusunun niyetini anlamak, uygun aracı seçmek ve aracın döndürdüğü deterministik sonucu doğal dile çevirmekle görevlidir:

```
Kullanici Sorgusu -> Intent Detection -> Dogru Deterministik Arac -> Tool Result -> Grounded LLM Response
```

c) LangGraph Mimarisi

Agent akışının kontrolü LangGraph ile sağlanmaktadır; bu sayede farklı sorgu türleri kontrollü şekilde ilgili işlem adımlarına (node) yönlendirilir:

```
Kullanici Sorgusu -> Intent Parser -> Router
+-----+-----+
      Finansman      Kampanya
(Bilgi/Karsilastir/Oner) (Bilgi/Karsilastir)
+-----+-----+
      Kaynakli (Grounded) Yanit -> Kullanici
```

d) Model Yaşam Döngüsü Yönetimi

İki büyük modelin (agent ve extractor) aynı anda GPU belleğinde tutulması gereksiz bellek kullanımına yol açabileceğinden model yaşam döngüsü yönetimi geliştirilmiştir: extractor kullanılacağı zaman

agent modeli bellekten çıkarılabilmekte (unload), extraction tamamlandıktan sonra extractor da bellekten çıkarılabilmektedir. Bu sayede GPU kaynakları daha verimli kullanılmaktadır.

e) Konuşma Durumu ve Oturum İzolasyonu

Sistem yalnızca tek mesajlık sorgular için tasarlanmamıştır; finansman ve kampanya için ayrı konuşma durumu (state) yapıları geliştirilmiştir. Örneğin kullanıcı “Kuveyt Türk araç finansmanı hakkında bilgi ver” dedikten sonra “Peki vadesi?” diye sorabilir; sistem önceki bağlamdan banka ve ürün bilgisini koruyarak ikinci soruyu yorumlar.

Birden fazla kullanıcının birbirinin konuşma durumuna erişmemesi için oturum izolasyonu geliştirilmiştir: state capture, state restore, validation ve session store mekanizmaları oluşturulmuş; eş zamanlı erişimler için RLock kullanılmış ve iki farklı oturumun birbirinden bağımsız çalıştığı test edilmiştir.

f) Deterministik Kural / Araç Katmanı (Tool Layer)

- Finansman Tool ve Kampanya Tool, veriye deterministik biçimde erişen yazılım bileşenleridir.
- Bu katman; banka çözümleme, ürün filtreleme, kriter bazlı karşılaştırma, aktif kampanya kontrolü, eksik veri kontrolü ve kaynak toplama işlemlerini gerçekleştirir.
- Finansal karar mantığı bu katmanda, kod düzeyinde tanımlanmıştır; büyük dil modelinin prompt'una gömülü değildir.

6. Farklı Bankalara Ait Benzer Ürünlerin Nasıl Karşılaştırıldığının Açıklaması

Kısaca: Bankaları tek bir “en iyi” puanla sıralamıyoruz; her özelliği (oran, vade, tutar vb.) ayrı ayrı karşılaştırıp kararı kullanıcıya bırakıyoruz. Geniş kapsamlı kampanya sorgularında ise yüzlerce kaydı özetleyerek yapay zekanın boğulmasını önliyoruz.

Finansman Sorgulama Motoru

Master veri kümesi oluşturulduktan sonra finansman kayıtları için deterministik bir sorgulama motoru geliştirilmiştir. Motor; banka, finansman kategorisi, ürün, finansman tutarı, oran, vade, taksit ve masraf gibi alanlar üzerinden sorgulama yapabilmektedir. Örneğin “Kuveyt Türk araç finansmanı” sorgusu önce ilgili bankaya ve ürün kategorisine resolve edilir (çözömlenir), ardından ilgili kayıtlar master veri kümesinden alınır. LLM'in görevi burada veriyi bulmak değil, bulunan deterministik veriyi kullanıcı için okunabilir hâle getirmektir.

Finansman Karşılaştırma Motoru

Karşılaştırmalar tek bir genel puan üzerinden yapılmaz; kriterler ayrı ayrı değerlendirilir:

- Kâr Payı Oranı
- Finansman Oranı
- Finansman Tutarı
- Vade
- Taksit Sayısı
- Masraf Bilgisi

Örneğin “Tüm bankaların araç finansmanlarını karşılaştıır” sorgusunda sistem ilgili bankaların araç finansmanı ürünlerini bulur; her kriter için bankadaki değer, kullanılan ürün, eksik veri, eşitlik durumu ve kriter bazında öne çıkan banka belirlenir. Sistem “X bankası genel olarak en iyidir” şeklinde kapalı bir skor üretmez. Bunun yerine şu türde açıklanabilir sonuçlar üretir:

- “Finansman oranında bu bankalar eşit.”
- “Taksit sayısında bu banka öne çıkıyor.”
- “Bu bankada veri belirtilmemiş.”

Bu yaklaşım, finansal karar destek sisteminde şeffaflık ve açıklanabilirlik sağlamak amacıyla tercih edilmiştir.

Örnek: Taşıt Finansmanı Karşılaştırması

Kriter	Kuveyt Türk	Ziraat Katılım
Finansman Oranı	%70	Belirtilmemiş
Vade	48 ay	48 ay

Çoklu Banka Karşılaştırma Arayüzü

İki veya üç banka karşılaştırılırken detaylı karşılaştırma görünümü kullanılır. Daha fazla banka olduğunda yatayda aşırı büyüyen klasik tablo kullanıcı deneyimini olumsuz etkilediği için arayüz daha kompakt hâle getirilmiştir: kriter başlıkları, banka değerleri, öne çıkan değerler, eşitlikler, “Belirtilmemiş” alanları ve masraf açıklamaları ayrı görsel bloklar hâlinde sunulur. Masraf bilgisi sayısal bir skor olarak değerlendirilmez; açıklamalı bir alan olarak gösterilir.

Kampanya Motoru

Finansman kayıtlarından ayrı olarak 414 kampanya için özel bir kampanya motoru geliştirilmiştir. Kampanyalar için banka, kategori, tür, avantaj, süre, hedef kitle, koşullar, tarih durumu ve kaynak bilgileri kullanılır. Sistem kampanyaları ayrıca zamansal olarak aktif, yaklaşan, sona ermiş veya tarih durumu doğrulanamayan şekilde sınıflandırabilmektedir.

Geniş Kampanya Sorgularının Yönetimi

“Aktif kampanyaları tüm bankalarda göster” gibi bir sorgu yüzlerce kayıt döndürebilmektedir; bir testte deterministik kampanya motoru **296 kampanya** bulmuştur. Ham deterministik sonuç yaklaşık **1.500.000 karakter** uzunluğunda bir içerik oluşturabildiğinden, bunun tamamının LLM'in bağlam penceresine verilmesi verimsiz hâle gelmiştir.

Bunun için bir **Broad Campaign Grounding Compaction Guard** (geniş kampanya sorguları için bağlam sıkıştırma katmanı) geliştirilmiştir. Bu sistem geniş sorgularda toplam kampanya sayısını, banka bazlı dağılımı ve her bankadan temsilî örnekleri LLM'e gönderirken, asıl deterministik kayıtları ve kaynakları korumaktadır. Böylece ~1.500.000 karakterlik veri, yaklaşık **23.000 karaktere** indirilebilmiş; LLM bağlam taşması ve cevap üretim hataları belirgin biçimde azaltılmıştır.

7. Projenin Çalıştırılması için Adım Adım Talimatlar

Kısaca: Harput Finans'ın ana uygulaması Google Colab / Jupyter Notebook ortamında çalışacak şekilde hazırlanmıştır. Yapay zeka modelleri Ollama üzerinden yerel olarak çalıştırılmaktadır.

Ön Gereksinimler

- Python 3.10 veya üzeri
- Git
- Ollama
- Google Colab veya Jupyter Notebook

1. Depoyu Klonlama

```
git clone https://github.com/HarputAI-Teknofest2026/harput-finans.git
cd harput-finans
```

2. Sanal Ortam Oluşturma

```
python -m venv .venv
```

macOS / Linux:

```
source .venv/bin/activate
```

Windows:

```
.venv\Scripts\activate
```

3. Python Bağımlılıklarını Yükleme

```
pip install -r requirements.txt
```

4. Gerekli Tarayıcı Bileşenlerinin Kurulması

```
playwright install
```

5. Ollama Servisinin Hazırlanması

```
ollama serve
```

Ollama'nın çalışıp çalışmadığı şu komutla kontrol edilebilir:

```
ollama list
```

6. Gerekli Modellerin Hazırlanması

```
ollama pull qwen3-coder-next
```

```
ollama pull qwen3.5:35b-a3b-q8_0
```

7. Ana Uygulamanın Çalıştırılması

Ana uygulama notebooku:

```
colab dosyaları/uygulama dosyaları/Harput_Finanas.ipynb
```

Notebook hücreleri sırasıyla çalıştırıldığında;

- Master veri seti yüklenir.
- Yapay zeka modelleri hazırlanır.
- Finansman ve kampanya araçları oluşturulur.
- Intent/routing katmanı hazırlanır.
- LangGraph tabanlı ajan akışı başlatılır.
- FastAPI backend hazırlanır.

- Web arayüzü çalıştırılır.

Sistem varsayılan olarak Ollama'ya şu adres üzerinden erişir:

```
http://127.0.0.1:11434
```


8. Karşılaşılan Problemler ve Çözüm Yaklaşımları

Kısaca: Geliştirme sürecinde karşılaştığımız en büyük zorlukları ve bunlara bulduğumuz çözümleri bu bölümde özetliyoruz.

Problem	Çözüm Yaklaşımı
Aynı ifadenin farklı finansal anlamlara gelebilmesi (örn. “%80” finansman oranı mı kâr payı mı; “24 ay” vade mi taksit mi kampanya süresi mi) nedeniyle sabit regex kurallarının güvenilir çalışmaması.	Sabit regex yerine, metne özel ayrıştırma kodu üreten (program sentezi) bir yaklaşıma geçildi; her yeni cümle yapısı için manuel kural yazma ihtiyacı ortadan kaldırıldı.
LLM'in ürettiği kodun doğrudan çalıştırılmasının güvenlik riski taşıması.	AST tabanlı statik güvenlik analizi, izole subprocess ve süre sınırlı (timeout) çalıştırma mekanizması eklendi; üretilen koda asla doğrudan güvenilmedi.
Geniş kapsamlı kampanya sorgularında (296 kampanya, ~1.500.000 karakter) LLM bağlam sınırının aşılması ve verimsizlik.	Broad Campaign Grounding Compaction Guard geliştirilerek veri, temsili örnekler ve özet istatistiklerle ~23.000 karaktere indirildi.
Kampanya arama sisteminde doğal dil komut kelimelerinin (“hakkında”, “bilgi”, “ver”) yanlışlıkla kampanya arama terimi sayılması nedeniyle var olan kayıtların bulunamaması (örn. “Biletinial'da %20 indirim kampanyası hakkında bilgi ver” sorgusunun eşleşmemesi).	Doğal dil komut kelimeleri ile gerçek kampanya kimlik terimleri (banka adı, kampanya adı, oran gibi) birbirinden ayrıştırıldı.
“Al'a Sor” promptundaki “taksit bilgisini açıkla” ifadesinin resolver tarafından yanlışlıkla “sadece taksit avantajı olan kampanyaları getir” filtresi olarak yorumlanması; bu yüzden yalnızca indirim türünde olan bir kampanyanın (Biletinial) yanlışlıkla elenmesi.	İlgili prompt sadeleştirilerek, bir kampanyayı sormak ile belirli özellikte kampanya aramak arasındaki anlam karışıklığı giderildi.
Kullanıcının yerel sayı formatında girdiği tutarların (“500.000”) parser tarafından yanlış yorumlanma riski.	Tutarlar arayüzden parser'a noktasız biçimde (“500000”) gönderilecek şekilde düzenlendi.
Birden fazla kullanıcının aynı anda sistemi kullanması durumunda konuşma durumlarının (state) birbirine karışma riski.	Oturum bazlı state capture/restore/validation mekanizmaları ve eş zamanlı erişim için RLock kullanılarak oturum izolasyonu sağlandı; iki farklı oturumun bağımsız çalıştığı test edildi.
Uzun yanıtların, izin verilen token sınırı nedeniyle yarıda kesilmesi (truncation).	Yanıt önce 1536 token ile üretiliyor; çıktı yarım kalırsa 3072 token ile ikinci bir üretim deneniyor. Yarım kalan cevap kullanıcıya hiçbir zaman gösterilmiyor.
İki büyük modelin (agent + extractor) aynı anda GPU belleğinde tutulmasının gereksiz kaynak tüketimine yol açması.	Model yaşam döngüsü yönetimi eklendi; kullanılmayan model ihtiyaç anına kadar bellekten çıkarılıyor (unload).
Aktif bir konuşma bağlamı varken kullanıcı yalnızca banka adı yazdığında (örn. “Ziraat Katılım”), stateless intent parser bunu her zaman FOLLOW_UP olarak sınıflandırmıyordu; deterministik backend bank-switch işlemini desteklediği hâlde istek LangGraph'ın niyet katmanında yanlış yönlendiriliyordu. Bu, frozen multi-turn test setiyle (bkz. Bölüm 10) tespit edilen gerçek bir production hatasıydı.	Bağlamsal “yalın banka” FOLLOW_UP guard eklendi: aktif konuşma bağlamı varken mesaj yalnızca banka seçimiye, parser sonucu ne olursa olsun turn deterministik backend'e FOLLOW_UP olarak gönderiliyor. Boş bir oturumda aynı ifade hâlâ GENERAL_CHAT olarak kalıyor; patch dar tutuldu.

Tüm bankalarda arama yapılan ama sıfır sonuç dönen kampanya sorgularında (örn. bulunmayan bir kampanya adı), sistem doğru şekilde 0 kayıt döndürse de araç durumu READY / SHOW_ALL olarak kalıyordu; NOT_FOUND'a geçmiyordu. Bu da bu turda test edilen no-data güvenlik setiyle (bkz. Bölüm 10) yakalanan bir status-contract hatasıydı.	Explicit-all boş-sonuç durumu için patch eklendi: all_banks_requested + READY + SHOW_ALL + records=[] koşulu artık NOT_FOUND / NO_MATCH olarak işaretleniyor. Kayıt bulunan pozitif senaryolarda (örn. “tüm bankaların idfix kampanyalarını göster”) davranış değişmedi.
--	---

Açık Geliştirme Alanları

- Geliştirme/demo ortamında kullanılan Cloudflare Quick Tunnel geçici bir bağlantı olduğundan, tünel durduğunda veya adres geçersizleştğinde “Error 1033” hatası oluşabilir. Production için kalıcı domain, kalıcı sunucu ve kalıcı tünel/deployment mimarisine geçilmesi planlanmaktadır.
- Extraction ve agent katmanları artık sistematik olarak ölçülmüştür (bkz. Bölüm 10); ancak bu ölçüm setleri takım tarafından hazırlanmış frozen test setleridir, bağımsız/insan tarafından etiketlenmiş bir referans veri seti değildir. Veri setinin periyodik güncellenmesi de hâlâ manuel bir süreçtir.

9. Model Çıktılarının Örnekleri

Kısaca: Bu bölümde, sistemin gerçek bir metinden ve gerçek bir kullanıcı sorusundan ne ürettiğini somut örneklerle gösteriyoruz.

a) Bilgi Çıkarımı (Extraction) Örneği

Girdi: “Bireysel finansman kapsamında 300.000 TL finansman sunulmaktadır. Aylık kâr payı oranı %1,89 olup finansman 24 ay vadeli olarak kullanılabilir. Finansman Türk Lirası cinsindendir.”

```
{
  "finansman_tutari": ["300.000 TL"],
  "kar_payi_orani": ["%1,89"],
  "vade": ["24 ay"],
  "para_birimi": ["TL"],
  "finansman_orani": [],
  "taksit_sayisi": [],
  "masraf_bilgisi": [],
  "hedef_kitle": [],
  "kosullar": []
}
```

b) Agent / Tool-Calling Örneği

Kullanıcı sorusu: “Kuveyt Türk ile Ziraat Katılım araç finansmanlarını karşılaştır.”

- Adım 1 — Intent: PRODUCT_COMPARE
- Adım 2 — Parametreler: Kategori: Taşıt Finansmanı; Bankalar: Kuveyt Türk, Ziraat Katılım
- Adım 3 — Tool Çağrısı: İlgili bankaların aynı kategorideki ürün kayıtları master veri kümesinden filtrelenir.
- Adım 4 — Deterministik Sonuç: Kâr Payı, Finansman Oranı, Tutar, Vade, Taksit, Masraf alanlarını içeren karşılaştırma tablosu üretilir.
- Adım 5 — Doğal Dil Yanıtı: “Kuveyt Türk’ün ilgili taşıt finansmanı ürünlerinde azami finansman oranı %70 olarak belirtilirken, Ziraat Katılım için karşılaştırılabilir bir finansman oranı kaynaklarda belirtilmemiştir. Vade açısından ise her iki bankanın ilgili taşıt finansmanı ürünlerinde 48 aya kadar seçenek bulunmaktadır.”

c) Ürün/Kampanya Kartından “Al’a Sor” Örneği

Senaryo: Kullanıcı, Ziraat Katılım’ın Banka Veri Gezgini’nde listelenen “Akaryakıt Harcamalarınıza 400 TL Bankkart Lira!” kampanya kartındaki “Al’a Sor” butonuna tıklar.

Sistem otomatik olarak şu sorguyu oluşturur ve AI Asistan’a iletir:

```
Ziraat Katılım Akaryakıt Harcamalarınıza 400 TL
Bankkart Lira! kampanyasi hakkında detayli bilgi ver.
```

Agent, ilgili deterministik kampanya kaydını bulur ve kampanya avantajı, süresi, hedef kitlesi, koşulları ile kaynağını kullanıcıya sunar.

d) AI Asistan Yanıt Rozetleri

Kullanıcı “Kuveyt Türk araç finansmanı hakkında bilgi ver” dediğinde cevapla birlikte şu bilgi rozetleri gösterilir: “[Finansman]”, “[Kaynaklı Cevap]”, “[1 Kaynak]”. Karşılaştırmalarda ise “[Finansman Karşılaştırma]” ve “[Kriter Bazlı]” rozetleri kullanılır.

e) Kampanya Sorgusu Örneđi

Kullanıcı sorusu: *“Aktif kampanyaları tüm bankalarda göster.”*

Sistem yanıtı: 414 kampanya kaydı içinden 296 aktif kampanya filtrelendir, banka bazlı özetlenip temsili örnekler ve kaynak bağlantılarıyla birlikte sunulur.

10. Modelin Performansını Değerlendirmek için Kullanılan Yöntemler

Kısaca: Sistemi tek bir 'doğruluk yüzdesi' ile ölçmüyoruz; extraction, niyet anlama, veri getirme, sayısal doğrulama, çok turlu konuşma ve 'veri yok' güvenliği gibi ayrı katmanlarda ölçüyoruz. Bu bölümde her katmanın gerçek sonucunu ve ölçümün sınırlarını dürüstçe paylaşıyoruz.

Değerlendirme Yaklaşımı: Neden Tek Bir Doğruluk Sayısı Değil?

Harput Finans tek bir model değildir; içinde niyet ayrıştırıcı (intent parser), deterministik veri getirme (retrieval), finansman/kampanya araçları, kaynağa dayalı yanıt üretimi (grounded response), çok turlu bağlam yönetimi ve ayrı bir bilgi çıkarımı (extraction) sistemi bulunur. Bu nedenle “Harput Finans %X doğru” gibi tek bir cümle metodolojik olarak yanıltıcı olur. Değerlendirme; Extraction, Intent, Slot/Filter, Grounding, Retrieval, Multi-turn ve No-data olmak üzere ayrı katmanlara bölünmüştür. Bir katmanın %100 çıkması, sistemin tamamının %100 doğru olduğu anlamına gelmez.

1) Extraction Benchmark

Finansman/kampanya metninden 9 sabit alan çıkaran program sentezi yaklaşımı ölçülmüştür. 50 kayıt seçilmiştir (30 finansman + 20 kampanya), tüm 10 banka kapsamıştır. Harput 3 kez, iki direct-JSON taban çizgisi (baseline) de 3'er kez çalıştırılmıştır. Toplam Harput değerlendirmesi $50 \times 3 \times 9 = 1.350$ alan ölçümüdür. Her alan DOĞRU / KISMEN / YANLIŞ / TEKNİK_HATA olarak ayrıca sınıflandırılmıştır.

Metrik	Sonuç	Kanıt
Teknik Başarı	%78.00	117 / 150
Strict — Tüm Alanlar	%47.85	646 / 1.350
Weighted — Tüm Alanlar	%54.00	kısmi puan dahil
Strict — Yalnızca Başarılı Kayıtlar	%61.35	teknik başarılı alt küme
Weighted — Yalnızca Başarılı Kayıtlar	%69.23	teknik başarılı alt küme

Taban Çizgisi (Baseline) Karşılaştırması

Extraction, iki direct-JSON taban çizgisiyle karşılaştırılmıştır: model doğrudan JSON üretir, program sentezi kullanmaz.

Metrik	Harput	Qwen3.5 9B Direct JSON	Llama3.1 8B Direct JSON
Teknik Başarı	%78.00	%100.00	%98.00
Strict — Tümü	%47.85	%73.19	%60.30
Weighted — Tümü	%54.00	%81.78	%68.93
Strict — Başarılı Kayıtlar	%61.35	%73.19	%61.53
Weighted — Başarılı Kayıtlar	%69.23	%81.78	%70.33

Önemli sınırlama: yalnızca yöntem değil, modeller de değişmektedir. Bu nedenle “program sentezi mimarisi taban çizgisinden şu kadar kötü” gibi nedensel bir iddia yapılmamaktadır; fark yöneme ve modele birlikte bağlıdır.

Alan Bazında Sonuçlar (Strict Doğruluk, %)

Alan	Harput	Qwen3.5 9B	Llama3.1 8B
Kâr payı oranı	47.33	97.33	76.00
Finansman oranı	68.67	90.00	76.67
Finansman tutarı	41.33	83.33	59.33
Vade	36.67	72.67	70.67
Taksit sayısı	55.33	79.33	61.33
Masraf bilgisi	57.33	74.67	62.67
Hedef kitle	26.00	42.00	34.00
Para birimi	65.33	96.00	92.00
Koşullar	32.67	23.33	10.00

Extraction Tutarlılığı (Consistency)

Aynı kayıt 3 kez çalıştırılarak cevapların birbirine ne kadar benzediği ölçülmüştür. Bu, doğruluk değil, kararlılık (stability) ölçütüdür.

- Alan-değer ortalama ikili Jaccard benzerliği: %65.33
- Varlık (presence) tutarlılığı: %79.11
- 9 alanın da 3 çalıştırmada birebir aynı olduğu kayıt oranı: %4.00
- En stabil alan: finansman_orani (%90.80) · En düşük: koşullar (%36.93)

2) AI Asistan Katmanları

a) Intent Classification

50 soruluk sabit (frozen) bir test seti oluşturulmuş, 7 intent sınıfı ve production'daki gerçek parse_user_intent_guarded() fonksiyonu kullanılmış, tekrar deneme (retry) yapılmamıştır.

- Sonuç: %94.00 doğruluk (47/50), 0 teknik hata, ortalama ~0.52 sn parser gecikmesi.
- Hataların çoğu (3/3) PRODUCT_COMPARE sınıfında yoğunlaşmıştır.
- Bu sonuç yalnızca “intent classification accuracy” olarak adlandırılmıştır; “ajan doğruluğu” olarak sunulmamıştır.

b) Structured Filter / Slot Extraction

30 soruluk frozen set ile parser'ın 7 alanı (intent, banks, all_banks, topic, product_category, requested_fields, comparison_criteria) ölçülmüştür; toplam 172 slot skorlanmıştır.

Bir değerlendirme hatası ve düzeltilmesi: İlk evaluator, parse_user_intent_guarded() fonksiyonunun bir envelope döndürdüğünü hesaba katmamış; gerçek parser sonucu result["parsed"] içindeyken üst seviye alanları okumaya çalışmış ve anlamsız düşük skor üretmiştir. Bu ilk sonuç geçersiz sayılmıştır. Aynı frozen set, production sözleşmesine göre düzeltilmiş bir evaluator (V2) ile yeniden ölçülmüştür.

- Raw Slot Extraction Accuracy: %74.42 (128/172)
- Strict Exact Filter Match: %33.33 (10/30) — sorudaki tüm skorlanan slotların aynı anda birebir eşleşmesini gerektiren çok sert bir metriktir.

- Bazı “hatalar” aslında canonicalization farklarıydı (örn. kar_payi_orani vs “kâr payı oranı”, masraf_bilgisi vs “masraf”, kosullar vs “koşullar”); skor şişirmek için sonradan normalize edilmemiş, ham sonuç olduğu gibi raporlanmıştır.

c) Numeric Financial Grounding

30 tek-turlu soru, sistemin tam LangGraph akışı üzerinden çalıştırılmıştır. Amaç, cevabın genel kalitesi değil, cevapta söylenen finansal sayıların araç kanıtında (tool evidence) gerçekten bulunup bulunmadığıdır.

- İlk evaluator Türkçe para formatı ve “taksit sayısı vs. süre” ayırımında hata yapmış; agent yeniden çalıştırılmadan, kaydedilmiş cevaplar düzeltilmiş bir evaluator ile offline olarak yeniden skorlanmıştır.
- 30 cevabın 14'ünde ölçülebilir bir finansal sayı bulunmuştur; bu 14 cevapta toplam 92 sayısal değer tespit edilmiş ve 92/92'si araç kanıtı tarafından desteklenmiştir.
- Numeric Financial Grounding: %100 (92/92) — bu, “30 cevabın tamamı %100 doğru” anlamına gelmez; yalnızca telaffuz edilen sayıların kaynaksız uydurulmadığını gösterir.

3) Retrieval Benchmark

Master veri 116 finansman + 414 kampanya = 530 kayıttır. 24 frozen sorgu oluşturulmuştur: finance single/compare/all-bank, campaign single/compare/all-bank. Ground truth, production retrieval fonksiyonundan değil, master kayıtlar üzerinde bağımsız bir Python filtresiyle oluşturulmuştur.

V1 tamamen geçersiz ilan edildi: evaluator'da üç hata bulunmuştur — (1) production'ın extract_real_tool_result() yardımcı fonksiyonu yerine özel bir unwrap fonksiyonu yazılmıştı, (2) gerçek finance-info aracı get_product_info olmasına rağmen evaluator'ın bilinen-arac listesinde yoktu, (3) kampanya sonuçlarında ürün adı kampanya_adi olarak dönerken evaluator yalnızca urun_adi arıyordu. Bu yüzden sistem doğru kayıt döndürürken evaluator “kayıt yok” raporluyordu. Bu run'daki tüm recall/F1 değerleri kullanımdan kaldırılmıştır.

Gerçek sözleşme (contract) 4 sanity-case ile gözle doğrulanmıştır: finance single → get_product_info, finance compare → compare_products, campaign single → get_campaign_info, campaign compare → compare_campaigns. Finansman kayıt adı urun_adi, kampanya kayıt adı kampanya_adi'dir. Dört test de READY sonucu vermiştir.

V2, bu gerçek sözleşmeye göre yeniden kurulmuştur: test seti production'ın kendi tahminlerinden önce diske yazılıp SHA ile dondurulmuş; production'ın extract_real_tool_result() fonksiyonu kullanılmış; bağımsız ground truth korunmuş; kategori genelinde “araç” yerine “taşıt” kullanılmıştır. V2, yeni bir ground-truth semantiği sorunu ortaya çıkarmıştır: Türkiye Finans'taki iki TICARI_TASIT kaydı, generic “taşıt” ground truth'una yanlışlıkla dahil edilmişti; production ise generic sorguda TICARI_TASIT'ı bilinçli olarak dışarıda bırakıyordu (yalnızca açıkça “ticari taşıt” denildiğinde getiriyordu). Yani 28 kayıt dönüp 30 beklenmesi bir retrieval hatası değil, bir ground-truth hatasıydı.

V3, production'ı yeniden çalıştırmadan, kaydedilmiş V2 tahminlerinin yalnızca ground-truth semantiği düzeltilerek offline yeniden skorlanmasıdır (yalnızca 2 sorunun etkilenmesine izin verilmiştir).

Metrik	Sonuç
Micro Precision / Recall / F1	%100 / %100 / %100 (TP=130, FP=0, FN=0)
Exact Retrieved Set Match	%100 (24/24)
Retrieved-record Source Integrity	%100 (130/130)

Ayrı bir sanity testinde “Türkiye Finans ticari taşıt finansmanlarını göster” sorgusu çalıştırılmış ve o iki ürünün açık ticari sorguda 2/2 döndüğü doğrulanmıştır — yani sistem kayıtları kaybetmiyordu, yalnızca generic sorgu kapsamında bilinçli olarak dışlıyordu.

4) Multi-turn Context Değerlendirmesi

Önce benchmark oluşturmadan geliştirme sanity testleri yapılmıştır (finans banka seçimi, finans alan takibi, kampanya banka seçimi, iç içe geçmiş iki oturum izolasyonu). İlk üçü çalışmış, ancak “Dünya Katılım Hepsiburada kampanyalarını göster” → “peki taksit sayısı?” takibi NOT_FOUND döndürmüştür.

Kök neden: kampanya alan alt sisteminde “taksit_sayısı” diye bir istenen-alan yoktu; taksit zaten kampanya tarafında TAKSIT fayda-türü olarak modellenmişti. Bu yüzden soru, önceki bağlamı taşımadan taze bir sorgu gibi işleniyordu. Kampanya bağlamsal iyileştirme (refinement) yaması eklenerek aktif kampanya kapsamı + kısa yapılandırılmış bir iyileştirme geldiğinde önceki kapsamla birleştirilmesi sağlanmıştır.

Yama sonrası gerçek Multi-turn V1 frozen benchmark'ı oluşturulmuştur: 10 senaryo, toplam 77 bağlam kontrolü (finans banka seçimi, vade/masraf takibi, banka değiştirme, kampanya banka seçimi, avantaj/süre takibi, taksit iyileştirme, kampanya alan+banka değiştirme, iç içe oturum izolasyonu).

Metrik	Düzeltilme Öncesi (V1)	Düzeltilme Sonrası (V2)
Context Check Accuracy	%85.71 (66/77)	%100.00 (77/77)
Scenario Exact Success	%80.00 (8/10)	%100.00 (10/10)

V1'in yakaladığı gerçek hata: finance_bank_switch ve campaign_field_bank_switch senaryoları başarısız olmuştur. Aktif bir konuşma bağlamı varken kullanıcı yalnızca banka adı yazdığında (örn. “Ziraat Katılım”), stateless intent parser bunu her zaman FOLLOW_UP saymıyordu; oysa deterministik backend bank-switch işlemini zaten destekliyordu. Sorun backend değil, LangGraph'ın niyet kapısıydı (bkz. Bölüm 8). Aynı frozen set, aynı SHA ile post-fix olarak yeniden çalıştırılmış ve pre-fix sonuçlar audit trail olarak saklanmıştır.

5) No-data Behavior Değerlendirmesi

12 frozen vaka oluşturulmuştur: 4 finans banka-yok, 4 kampanya banka-yok, 2 global kampanya NOT_FOUND, 2 kampanya-karşılaştırma NOT_FOUND. Ground truth, tahminler görülmeden önce doğrudan master veri üzerinden doğrulanmıştır. Her vakada 9 ayrı güvenlik koşulu kontrol edilmiştir: doğru yönlendirme, doğru araç, doğru no-data durumu, istenen kayıt sızıntısı olmaması, strict NOT_FOUND'da kayıt/kaynağın boş olması, kaynak bütünlüğü, cevapta URL sızıntısı olmaması, desteklenmeyen sayısal iddia olmaması ve cevabın gerçekten “veri yok” anlamı taşıması.

Metrik	Düzeltilme Öncesi (V1)	Düzeltilme Sonrası (V2)
Safety Check Accuracy	%99.07 (107/108)	%100.00 (108/108)
Case Exact Success	%91.67 (11/12)	%100.00 (12/12)

V1'in tek hatası: “Tüm bankaların Qorvexa kampanyalarını göster” vakasıydı. Sistem aslında doğru cevap veriyordu (0 kayıt, 0 kaynak, Qorvexa'nın bulunmadığını açıkça belirtiyordu) fakat araç durumu NOT_FOUND yerine READY kalıyordu. Kök neden: kampanya banka-seçimi analizöründe explicit-all dalı, available_banks boş mu kontrolünden önce çalışıyor ve sıfır kayıt olsa bile koşulsuz READY/SHOW_ALL döndürüyordu (bkz. Bölüm 8). Pozitif regresyon (“tüm bankaların idexif kampanyalarını göster”, 5 kampanya) hâlâ doğru şekilde READY/SHOW_ALL döndürmüştür; olağan global NOT_FOUND davranışı bozulmamıştır.

- Post-fix sonucunda: 0 kayıt sızıntısı, 0 geçersiz kaynak, 0 desteklenmeyen sayısal iddia, 0 URL sızıntısı.

Final Sonuç Özeti

Değerlendirme Katmanı	Final Sonuç
Extraction — Teknik Başarı	%78.00 (117/150)
Extraction — Strict (Tüm Alanlar)	%47.85
Extraction — Weighted (Tüm Alanlar)	%54.00
Extraction — Strict (Başarılı Kayıtlar)	%61.35
Extraction — Weighted (Başarılı Kayıtlar)	%69.23
Extraction — Alan-değer Tutarlılığı	%65.33 (Jaccard)
Intent Classification	%94.00 (47/50)
Raw Slot Extraction	%74.42 (128/172)
Strict Exact Filter Match	%33.33 (10/30)
Numeric Financial Grounding	%100 (92/92)
Retrieval Micro P / R / F1	%100 / %100 / %100
Retrieval Exact Set Match	%100 (24/24)
Retrieved-record Source Integrity	%100 (130/130)
Multi-turn Context — düzeltme öncesi / sonrası	%85.71 → %100
Multi-turn Scenario Exact — düzeltme öncesi / sonrası	%80 → %100
No-data Safety — düzeltme öncesi / sonrası	%99.07 → %100
No-data Case Exact — düzeltme öncesi / sonrası	%91.67 → %100

Değerlendirmenin Sınırları

- Intent, slot/filter ve grounding test setleri, bağımsız insan tarafından etiketlenmiş bir referans veri seti değil, takım tarafından hazırlanmış frozen test setleridir.
- Retrieval testi bağımsız türetilmiş master-veri ground truth'u kullanır; ancak V3 skoru, açıkça belgelenmiş bir post-hoc evaluator semantiği düzeltmesi içerir.
- Multi-turn ve no-data post-fix sonuçları, pre-fix ile aynı frozen setler üzerinde yapılan regresyon ölçümleridir.
- Bu nedenle proje, tek bir genel doğruluk değeri yerine katman bazında ayrı metrikler raporlamaktadır.

En önemli bulgu yalnızca yüksek skorlar değildir: değerlendirme süreci gerçekten production hataları bulmuştur. Retrieval'da evaluator hatalarını sistem hatası olarak raporlamadık; multi-turn'de iki gerçek bağlam hatası, no-data'da bir durum-sözleşmesi (status-contract) hatası yakalandı; düzeltme sonrası sorular değiştirilmeden aynı frozen setlerle regresyon testi yapıldı.

Kritik raporlama kuralımız: “Harput Finans genel doğruluğu %100” demiyoruz. %100 çıkan retrieval, grounding, multi-turn ve no-data sonuçlarının her birinin kendi kapsamı ayrı ayrı belirtilmiştir.